FIAP

EcoTrack API

Testes automatizados com BDD: cenários em Gherkin, testes de API e contratos em JSON Schema

Aluno	**Paulo Gabriel de Souza**
RM	**566228**
Repositório	**github.com/psouzah12/ecotrack-testes**
API testada	**github.com/psouzah12/ecotrack-devops**
Data	**Novembro de 2026**

1. Visão geral

O EcoTrack é a API REST em Java 21 e Spring Boot que desenvolvi nas sprints anteriores para registrar dados ambientais de empresas: cadastro de empresas, indicadores de água, energia, resíduos e CO2, ações ESG e relatórios. Na entrega de DevOps ela ganhou Docker, pipeline de CI/CD e uma imagem publicada no GitHub Container Registry.

Nesta entrega escrevi os testes automatizados da API em BDD. Os cenários estão em Gherkin, em português, e cada passo vira uma chamada HTTP à API rodando com o Oracle no Docker Compose. Não há mock: o que está sendo testado é a versão 1.1.0 da imagem publicada no GHCR, fixada pela tag `sha-92e76ea`, o mesmo build que foi para staging e produção.

Pedido no enunciado	O que fiz
No mínimo 3 cenários BDD em Gherkin, positivos e negativos	56 cenários em 6 arquivos .feature: 33 de caminho feliz e 23 de falha
Testes de todas as APIs: status code, corpo JSON e contrato	As 21 rotas da API, mais health e info. Todo cenário confere o status da resposta que está testando e, quando ela tem corpo, valida o contrato em JSON Schema (13 contratos). Os cenários de cada funcionalidade também conferem os valores dos campos
Execução simples e documentada, local ou em CI	`scripts/executar-testes.sh` sobe tudo e roda os testes só com Docker. O workflow do GitHub Actions faz o mesmo e publica o relatório; o run #3 passou em 1m37s
Integração com ESG	Indicadores ambientais, ações de reciclagem e eficiência energética, mensagens de erro claras, controle de acesso, integridade e rastreabilidade (seção 5)

> Na execução de 06/11/2026, às 22h00, os 56 cenários e os 436 passos passaram em 21 segundos, contando a criação e a limpeza dos dados (o Cucumber conta o hook de limpeza de cada cenário como passo). O ciclo inteiro, da subida do Oracle até o relatório, levou menos de 2 minutos, com as imagens Docker e as dependências do Maven já em cache. No GitHub Actions, o run #3 do repositório `psouzah12/ecotrack-testes` rodou os mesmos 56 cenários e passou.

2. Estrutura e configuração

O projeto de testes é separado da API. Ele só precisa de uma EcoTrack no ar: por padrão sobe a imagem publicada, mas pode apontar para outro endereço (o staging, por exemplo) ou usar uma imagem gerada na máquina.

estrutura do projeto

```
ecotrack-testes/
├── .github/workflows/testes-bdd.yml    pipeline do GitHub Actions
├── docker-compose.yml                  Oracle, API e o container dos testes
├── scripts/
│   ├── executar-testes.sh              sobe tudo, roda os cenários e derruba
│   └── ci/resumo-cucumber.sh           tabela de resultados no resumo do Actions
├── testes/
│   ├── Dockerfile                      imagem que roda os testes
│   ├── pom.xml
│   └── src/test/
│       ├── java/br/com/fiap/ecotrack/bdd/
│       │   ├── ExecutarCenariosTest.java   suíte que chama o Cucumber
│       │   ├── EmpresaSteps.java, IndicadorSteps.java, AcaoSteps.java,
│       │   │   RelatorioSteps.java, AcessoSteps.java, RespostaSteps.java
│       │   ├── Cenario.java                estado do cenário e chamadas HTTP
│       │   ├── Contrato.java               validação com JSON Schema
│       │   ├── Dados.java                  corpos das requisições
│       │   └── Hooks.java                  espera a API e apaga os dados criados
│       └── resources/
│           ├── features/                   6 arquivos .feature
│           ├── schemas/                    13 contratos e o comum.json
│           └── junit-platform.properties   configuração do Cucumber
├── evidencias/                         prints, logs e o relatório HTML do Cucumber
├── docs/                               este PDF
└── README.md
```

Como os testes funcionam

- A JUnit Platform chama o Cucumber pela classe `ExecutarCenariosTest`, e o Surefire roda essa classe no `mvn test`.
- Cada classe `*Steps.java` implementa os passos de uma parte da API. O PicoContainer entrega a todas elas o mesmo objeto `Cenario`, que guarda o usuário logado, a última resposta e os registros criados.
- As chamadas HTTP usam o REST Assured. Cada requisição e resposta fica anexada ao cenário no relatório HTML do Cucumber.
- Os passos "Dado" criam pela própria API os dados de que o cenário precisa, com o usuário ADMIN. No fim do cenário um hook apaga tudo, do último registro para o primeiro. Os cenários não dependem de ordem nem dos dados de exemplo, e o banco termina como começou.
- Antes do primeiro cenário, outro hook espera o `/actuator/health/readiness` responder 200, por até 3 minutos, porque o Oracle demora para subir.

Configuração do Cucumber

testes/src/test/resources/junit-platform.properties

```
cucumber.glue=br.com.fiap.ecotrack.bdd
cucumber.plugin=pretty, summary, html:target/cucumber/relatorio.html, json:target/cucumber/cucumber.json,
junit:target/cucumber/cucumber.xml
cucumber.publish.quiet=true
cucumber.snippet-type=camelcase
# o relatório do Surefire mostra o nome do cenário em vez do número da linha
cucumber.junit-platform.naming-strategy=long
```

testes/src/test/java/br/com/fiap/ecotrack/bdd/ExecutarCenariosTest.java

```
package br.com.fiap.ecotrack.bdd;

import org.junit.platform.suite.api.IncludeEngines;
import org.junit.platform.suite.api.SelectPackages;
import org.junit.platform.suite.api.Suite;

// roda todos os .feature de src/test/resources/features; a configuração do Cucumber
// fica em src/test/resources/junit-platform.properties
@Suite
@IncludeEngines("cucumber")
@SelectPackages("features")
public class ExecutarCenariosTest {
}
```

O plugin `pretty` mostra os passos no console, o `html` gera o relatório navegável, o `json` alimenta o resumo do GitHub Actions e o `junit` serve para qualquer ferramenta de CI que leia XML.

Dependências (pom.xml)

testes/pom.xml: versões

```
    <properties>
        <maven.compiler.release>21</maven.compiler.release>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

        <cucumber.version>7.34.9</cucumber.version>
        <junit.version>5.14.4</junit.version>
        <rest-assured.version>5.5.7</rest-assured.version>
        <json-schema-validator.version>1.5.9</json-schema-validator.version>
        <assertj.version>3.27.7</assertj.version>
    </properties>
```

testes/pom.xml: dependências de teste

```
io.cucumber:cucumber-java
io.cucumber:cucumber-junit-platform-engine
io.cucumber:cucumber-picocontainer
org.junit.platform:junit-platform-suite
io.rest-assured:rest-assured
com.networknt:json-schema-validator
org.assertj:assertj-core
org.slf4j:slf4j-nop
```

Docker

docker-compose.yml

```
# Ambiente dos testes BDD: Oracle, API EcoTrack e o container que roda os cenários.
#
#   docker compose up -d --wait oracle api   sobe só o banco e a API (porta 8080)
#   docker compose run --rm --build testes   roda os cenários contra a API
#   docker compose down -v                   derruba tudo e apaga o banco
#
# A API vem da imagem que o pipeline do projeto ecotrack-devops publicou no GHCR.
# A tag sha-92e76ea é fixa (a 1.1.0 é regravada a cada build da main), então
# os testes sempre rodam contra o mesmo build.
# Para testar outra versão: ECOTRACK_IMAGE=ecotrack-api:local docker compose ...
name: ecotrack-testes

services:
  oracle:
    image: gvenzl/oracle-free:23-slim
    environment:
      ORACLE_PASSWORD: ${ORACLE_ADMIN_PASSWORD:-oracle123}
      APP_USER: ECOTRACK
      APP_USER_PASSWORD: ${DB_PASSWORD:-ecotrack123}
    healthcheck:
      test: ["CMD", "healthcheck.sh"]
      interval: 10s
      timeout: 5s
      retries: 30
      start_period: 60s

  api:
    image: ${ECOTRACK_IMAGE:-ghcr.io/psouzah12/ecotrack-api:sha-92e76ea}
    depends_on:
      oracle:
        condition: service_healthy
    environment:
      APP_ENV: testes
      SPRING_DATASOURCE_URL: jdbc:oracle:thin:@oracle:1521/FREEPDB1
      SPRING_DATASOURCE_USERNAME: ECOTRACK
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD:-ecotrack123}
      ECOTRACK_ADMIN_USER: ${ECOTRACK_ADMIN_USER:-admin@ecotrack.com}
      ECOTRACK_ADMIN_PASSWORD: ${ECOTRACK_ADMIN_PASSWORD:-admin123}
      ECOTRACK_USER: ${ECOTRACK_USER:-user@ecotrack.com}
      ECOTRACK_USER_PASSWORD: ${ECOTRACK_USER_PASSWORD:-user123}
    ports:
      - "${APP_PORT:-8080}:8080"
    # o healthcheck vem da imagem (GET /actuator/health/readiness, que inclui o banco)

  testes:
    build: ./testes
    profiles: ["testes"]
    depends_on:
      api:
        condition: service_healthy
    environment:
      ECOTRACK_BASE_URL: http://api:8080
      ECOTRACK_ADMIN_USER: ${ECOTRACK_ADMIN_USER:-admin@ecotrack.com}
      ECOTRACK_ADMIN_PASSWORD: ${ECOTRACK_ADMIN_PASSWORD:-admin123}
      ECOTRACK_USER: ${ECOTRACK_USER:-user@ecotrack.com}
      ECOTRACK_USER_PASSWORD: ${ECOTRACK_USER_PASSWORD:-user123}
    volumes:
      # relatórios do Cucumber e do Surefire ficam na máquina em testes/target
      - ./testes/target:/testes/target
      - maven-repo:/root/.m2

volumes:
  maven-repo:
```

testes/Dockerfile

```
# Container que roda os cenários BDD contra a API (usado pelo docker-compose.yml).
# Os argumentos depois do nome do serviço vão para o Maven:
#   docker compose run --rm --build testes -Dcucumber.filter.tags=@negativo
# O --build refaz a imagem quando um .feature, .java ou schema muda.
FROM maven:3.9.9-eclipse-temurin-21

WORKDIR /testes

# dependências numa camada própria: só baixa de novo quando o pom.xml muda
COPY pom.xml .
RUN mvn -B -ntp dependency:go-offline

COPY src ./src

ENV ECOTRACK_BASE_URL=http://api:8080
ENTRYPOINT ["mvn", "-B", "-ntp", "test"]
```

scripts/executar-testes.sh

```
#!/usr/bin/env bash
# Roda os cenários BDD do começo ao fim, só com Docker: sobe o Oracle e a API,
# executa os testes no container e derruba o ambiente.
#
# Uso: scripts/executar-testes.sh [argumentos do Maven]
#   scripts/executar-testes.sh
#   scripts/executar-testes.sh -Dcucumber.filter.tags=@negativo
#   MANTER_AMBIENTE=1 scripts/executar-testes.sh    (não derruba no fim)
#
# Relatórios: testes/target/cucumber/relatorio.html, cucumber.json e cucumber.xml
set -euo pipefail
cd "$(dirname "$0")/.."

# derruba o ambiente também quando a subida falha ou a execução é interrompida
derrubar() {
  if [ "${MANTER_AMBIENTE:-0}" != 1 ]; then
    echo "==> Derrubando o ambiente"
    docker compose down -v
  fi
}
trap derrubar EXIT

echo "==> Subindo Oracle e API (na primeira vez leva uns 2 minutos)"
docker compose up -d --wait oracle api
docker compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.Status}}'
echo "==> API em http://localhost:${APP_PORT:-8080}"

echo "==> Rodando os cenários"
status=0
docker compose run --rm --build testes "$@" || status=$?

# o container roda como root: devolve testes/target para o usuário da máquina (importa no Linux)
docker compose run --rm --no-deps --entrypoint chown testes -R "$(id -u):$(id -g)" /testes/target

echo "==> Relatório em testes/target/cucumber/relatorio.html"
exit "$status"
```

Pipeline de CI

.github/workflows/testes-bdd.yml

```yaml
# Cenários BDD da API EcoTrack.
#
# Sobe o Oracle e a API (imagem publicada no GHCR) com o docker compose do
# repositório, roda os cenários com o Maven do runner e publica o relatório
# do Cucumber como artefato. A tabela de resultados aparece no resumo do run.
name: Testes BDD

on:
  push:
    branches: [main]
    paths-ignore: ["**.md", "docs/**", "evidencias/**"]
  pull_request:
    branches: [main]
  workflow_dispatch:
    inputs:
      tags:
        description: "Filtro de tags do Cucumber (ex.: @negativo). Se ficar vazio, roda todos os
cenários."
        required: false
        default: ""

permissions:
  contents: read

jobs:
  bdd:
    name: Cenários BDD e contratos
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v7

      - name: Configurar JDK 21
        uses: actions/setup-java@v6
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
          cache-dependency-path: testes/pom.xml

      - name: Subir Oracle e API
        run: |
          docker compose up -d --wait oracle api
          docker compose ps
          curl -s http://localhost:8080/actuator/info

      - name: Rodar os cenários
        working-directory: testes
        env:
          ECOTRACK_BASE_URL: http://localhost:8080
          TAGS: ${{ inputs.tags }}
        run: mvn -B -ntp test ${TAGS:+"-Dcucumber.filter.tags=$TAGS"}

      - name: Resumo dos cenários
        if: always()
        run: bash scripts/ci/resumo-cucumber.sh testes/target/cucumber/cucumber.json >>
"$GITHUB_STEP_SUMMARY"

      - name: Publicar relatório do Cucumber
        if: always()
        uses: actions/upload-artifact@v7
        with:
          name: relatorio-cucumber
          path: |
            testes/target/cucumber/
```

```
          testes/target/surefire-reports/
        retention-days: 14

    - name: Logs da API
      if: failure()
      run: docker compose logs --tail 200 api

    - name: Derrubar o ambiente
      if: always()
      run: docker compose down -v
```

3. Cenários em Gherkin

Os cenários estão em português (`# language: pt`). Cada funcionalidade começa com o formato "Como / Quero / Para", que diz quem usa aquela parte da API e com qual objetivo. Os passos falam a língua do negócio ("que a empresa registrou 7800 kWh de ENERGIA") e o detalhe técnico fica no Java.

Usei `Contexto` para os passos comuns a todos os cenários de um arquivo e `Esquema do Cenário` com `Exemplos` quando o mesmo comportamento vale para vários valores. Os números seguem o padrão brasileiro (`1320,5`), porque o Cucumber lê o `{bigdecimal}` com vírgula decimal em arquivos em português.

Tag	Cenários	Uso
`@positivo` / `@negativo`	33 / 23	caminho feliz e falha
`@governanca`	16	permissão, integridade dos dados e rastreabilidade
`@inclusao`	4	mensagens de erro que dizem qual campo corrigir
`@eficiencia`	4	tempo de resposta das consultas públicas
`@empresas` , `@indicadores` , `@acoes` , `@relatorios` , `@acesso` , `@monitoramento`		uma por funcionalidade, para rodar só uma parte

3.1 Cadastro de empresas `empresas.feature`

Indicadores e ações sempre pertencem a uma empresa, então comecei pelo cadastro. Os cenários passam pelo CRUD inteiro e pelas duas regras de governança do cadastro: o CNPJ não se repete, e uma empresa com indicadores não pode ser apagada, porque o histórico ambiental dela se perderia.

Cenário	Tipo	O que confere
Cadastrar uma empresa	positivo	201, contrato *empresa*, UF salva em maiúscula e empresa presente na listagem
Consultar uma empresa pelo id	positivo	200 e os mesmos dados enviados no cadastro
Listar as empresas	positivo	200, contrato *lista-empresas* e a empresa criada na lista
Mudar o setor e a cidade de uma empresa	positivo	200 e os dois campos alterados
Excluir uma empresa sem registros ligados a ela	positivo	204 e, em seguida, 404 com o contrato *erro* na consulta
Recusar um CNPJ que já está cadastrado	negativo	400 com a mensagem da regra de negócio

Cenário	Tipo	O que confere
Impedir a exclusão de uma empresa que tem indicadores	negativo	409, violação de integridade
Consultar uma empresa que não existe	negativo	404 com o id na mensagem
Consultar uma empresa com id que não é número	negativo	400 dizendo qual parâmetro está errado
Dizer qual campo do cadastro está errado (3 exemplos)	negativo	400 com a lista de erros, um por campo

testes/src/test/resources/features/empresas.feature

```
# language: pt
@empresas
Funcionalidade: Cadastro de empresas
  Como analista ESG
  Quero manter o cadastro das empresas acompanhadas pelo EcoTrack
  Para lançar os indicadores e as ações ESG de cada uma

  Contexto:
    Dado que estou autenticado como ADMIN

  @positivo
  Cenário: Cadastrar uma empresa
    Quando eu cadastro a empresa:
      | nome   | Reciclar Embalagens LTDA |
      | setor  | Reciclagem               |
      | cidade | Santos                   |
      | estado | sp                       |
    Então a resposta tem status 201
    E o corpo segue o contrato "empresa"
    E o campo "nome" vale "Reciclar Embalagens LTDA"
    E o campo "estado" vale "SP"
    E a empresa aparece na listagem de empresas

  @positivo
  Cenário: Consultar uma empresa pelo id
    Dado que existe uma empresa cadastrada
    Quando eu consulto a empresa
    Então a resposta tem status 200
    E o corpo segue o contrato "empresa"
    E o corpo traz os dados da empresa cadastrada

  @positivo
  Cenário: Listar as empresas
    Dado que existe uma empresa cadastrada
    Quando eu consulto "/api/empresas"
    Então a resposta tem status 200
    E o corpo segue o contrato "lista-empresas"
    E a empresa cadastrada está na lista

  @positivo
  Cenário: Mudar o setor e a cidade de uma empresa
    Dado que existe uma empresa cadastrada
    Quando eu altero a empresa para o setor "Energia solar" e a cidade "Campinas"
    Então a resposta tem status 200
    E o corpo segue o contrato "empresa"
    E o campo "setor" vale "Energia solar"
    E o campo "cidade" vale "Campinas"
```

```
  @positivo
  Cenário: Excluir uma empresa sem registros ligados a ela
    Dado que existe uma empresa cadastrada
    Quando eu excluo a empresa
    Então a resposta tem status 204
    E a empresa não é mais encontrada

  @negativo @governanca
  Cenário: Recusar um CNPJ que já está cadastrado
    Dado que existe uma empresa cadastrada
    Quando eu cadastro outra empresa com o mesmo CNPJ
    Então a resposta tem status 400
    E o corpo segue o contrato "erro"
    E a mensagem de erro é "Já existe uma empresa cadastrada com este CNPJ."

  @negativo @governanca
  Cenário: Impedir a exclusão de uma empresa que tem indicadores
    Dado que existe uma empresa cadastrada
    E que a empresa registrou 4200 "kWh" de "ENERGIA"
    Quando eu excluo a empresa
    Então a resposta tem status 409
    E o corpo segue o contrato "erro"
    E a mensagem de erro é "Operação viola integridade dos dados. Verifique se o registro possui
dependências."

  @negativo
  Cenário: Consultar uma empresa que não existe
    Quando eu consulto a empresa de id 999999
    Então a resposta tem status 404
    E o corpo segue o contrato "erro"
    E a mensagem de erro é "Empresa não encontrada com ID: 999999"

  @negativo
  Cenário: Consultar uma empresa com id que não é número
    Quando eu consulto "/api/empresas/abc"
    Então a resposta tem status 400
    E o corpo segue o contrato "erro"
    E a mensagem de erro é "Valor inválido para o parâmetro 'id'."

  @negativo @inclusao
  Esquema do Cenário: Dizer qual campo do cadastro está errado
    Quando eu cadastro uma empresa com "<campo>" igual a "<valor>"
    Então a resposta tem status 400
    E o corpo segue o contrato "erros-validacao"
    E o campo "<campo>" tem o erro "<mensagem>"

    Exemplos:
      | campo  | valor          | mensagem                                    |
      | cnpj   | 12345678000190 | O CNPJ deve estar no formato XX.XXX.XXX/XXXX-XX |
      | estado | SPX            | O estado deve conter 2 caracteres           |
      | nome   |                | O nome é obrigatório                        |
```

3.2 Indicadores ambientais `indicadores.feature`

Aqui ficam o consumo de água e de energia, os resíduos gerados e as emissões de CO2, lançados mês a mês. O esquema de cenário registra um indicador de cada tipo, com valores decimais no padrão brasileiro (1320,5 m³ de água, 95,75 tCO2e).

Cenário	Tipo	O que confere
Registrar o indicador do mês (4 exemplos)	positivo	201, contrato *indicador*, tipo, valor e nome da empresa
Consultar um indicador pelo id	positivo	200, valor e unidade
Listar os indicadores de uma empresa	positivo	200, contrato de lista e quantidade certa
Filtrar os indicadores de resíduos	positivo	200, todos os itens do tipo RESIDUOS e o indicador criado na lista
Corrigir um valor lançado errado	positivo	200 e o valor novo
Excluir um indicador	positivo	204 e, em seguida, 404 com o contrato *erro* na consulta
Recusar valor negativo	negativo	400 com o erro no campo *valor*
Recusar indicador de uma empresa que não existe	negativo	404
Recusar um tipo de indicador desconhecido no filtro	negativo	400 dizendo que o parâmetro *tipo* é inválido
Consultar um indicador que não existe	negativo	404, contrato *erro* e a mensagem com o id

testes/src/test/resources/features/indicadores.feature

```
# language: pt
@indicadores
Funcionalidade: Indicadores ambientais
  Como analista ESG
  Quero registrar o consumo de água e energia, os resíduos gerados e as emissões de CO2
  Para acompanhar mês a mês o impacto ambiental de cada empresa

  Contexto:
    Dado que estou autenticado como ADMIN
    E que existe uma empresa cadastrada

  @positivo
  Esquema do Cenário: Registrar o indicador do mês
    Quando eu registro <valor> "<unidade>" de "<tipo>" referente a "2026-10-01"
    Então a resposta tem status 201
    E o corpo segue o contrato "indicador"
    E o campo "tipo" vale "<tipo>"
    E o campo "valor" vale <valor>
    E o campo "empresaNome" vale o nome da empresa

    Exemplos:
      | tipo        | valor  | unidade |
      | AGUA        | 1320,5 | m3      |
      | ENERGIA     | 7800   | kWh     |
      | RESIDUOS    | 410    | kg      |
      | EMISSAO_CO2 | 95,75  | tCO2e   |

  @positivo
  Cenário: Consultar um indicador pelo id
    Dado que a empresa registrou 7800 "kWh" de "ENERGIA"
    Quando eu consulto o indicador
```

```
  Então a resposta tem status 200
  E o corpo segue o contrato "indicador"
  E o campo "valor" vale 7800
  E o campo "unidadeMedida" vale "kWh"

@positivo
Cenário: Listar os indicadores de uma empresa
  Dado que a empresa registrou 1320,5 "m3" de "AGUA"
  E que a empresa registrou 7800 "kWh" de "ENERGIA"
  Quando eu listo os indicadores da empresa
  Então a resposta tem status 200
  E o corpo segue o contrato "lista-indicadores"
  E a lista tem 2 itens

@positivo
Cenário: Filtrar os indicadores de resíduos
  Dado que a empresa registrou 410 "kg" de "RESIDUOS"
  Quando eu listo os indicadores do tipo "RESIDUOS"
  Então a resposta tem status 200
  E o corpo segue o contrato "lista-indicadores"
  E todos os itens têm "tipo" igual a "RESIDUOS"
  E o indicador registrado está na lista

@positivo
Cenário: Corrigir um valor lançado errado
  Dado que a empresa registrou 7800 "kWh" de "ENERGIA"
  Quando eu corrijo o valor do indicador para 6950,4
  Então a resposta tem status 200
  E o corpo segue o contrato "indicador"
  E o campo "valor" vale 6950,4

@positivo
Cenário: Excluir um indicador
  Dado que a empresa registrou 1320,5 "m3" de "AGUA"
  Quando eu excluo o indicador
  Então a resposta tem status 204
  E o indicador não é mais encontrado

@negativo
Cenário: Recusar valor negativo
  Quando eu registro -10 "kWh" de "ENERGIA" referente a "2026-10-01"
  Então a resposta tem status 400
  E o corpo segue o contrato "erros-validacao"
  E o campo "valor" tem o erro "O valor deve ser maior ou igual a zero"

@negativo
Cenário: Recusar indicador de uma empresa que não existe
  Quando eu registro um indicador para a empresa de id 999999
  Então a resposta tem status 404
  E o corpo segue o contrato "erro"
  E a mensagem de erro é "Empresa não encontrada com ID: 999999"

@negativo
Cenário: Recusar um tipo de indicador desconhecido no filtro
  Quando eu listo os indicadores do tipo "PLASTICO"
  Então a resposta tem status 400
  E o corpo segue o contrato "erro"
  E a mensagem de erro é "Valor inválido para o parâmetro 'tipo'."

@negativo
Cenário: Consultar um indicador que não existe
  Quando eu consulto "/api/indicadores/999999"
  Então a resposta tem status 404
  E o corpo segue o contrato "erro"
  E a mensagem de erro é "Indicador ESG não encontrado com ID: 999999"
```

3.3 Ações ESG `acoes-esg.feature`

Os cenários seguem o ciclo de status de uma ação ESG (planejada, em andamento, concluída e cancelada) com exemplos que uma empresa faria de verdade: coleta seletiva, troca de lâmpadas por LED, reúso de água, plantio de mudas. Também conferem que a API recusa status inventado e cadastro incompleto.

Cenário	Tipo	O que confere
Planejar uma ação de coleta seletiva	positivo	201, contrato *acao*, categoria, status e empresa
Atualizar o andamento de uma ação (3 exemplos)	positivo	200 e o status novo (PATCH)
Consultar uma ação pelo id	positivo	200, título e impacto
Listar as ações de uma empresa	positivo	200 e quantidade certa
Filtrar as ações concluídas	positivo	200, todos os itens CONCLUIDA e a ação criada na lista
Ampliar uma ação já cadastrada	positivo	200, título e impacto novos (PUT)
Excluir uma ação	positivo	204 e, em seguida, 404 com o contrato *erro* na consulta
Recusar um status que não existe	negativo	400, corpo inválido
Apontar todos os campos obrigatórios que faltam	negativo	400 com quatro erros, um por campo
Mudar o status de uma ação que não existe	negativo	404
Consultar uma ação que não existe	negativo	404, contrato *erro* e a mensagem com o id

`testes/src/test/resources/features/acoes-esg.feature`

```
# language: pt
@acoes
Funcionalidade: Ações ESG
  Como gestor de sustentabilidade
  Quero planejar ações ESG e atualizar o andamento de cada uma
  Para saber o que já foi feito e qual impacto esperar

  Contexto:
    Dado que estou autenticado como ADMIN
    E que existe uma empresa cadastrada

  @positivo
  Cenário: Planejar uma ação de coleta seletiva
    Quando eu cadastro a ação:
      | titulo          | Coleta seletiva no refeitório                        |
      | descricao       | Lixeiras separadas e coleta semanal pela cooperativa |
      | categoria       | RECICLAGEM                                           |
      | status          | PLANEJADA                                            |
      | impactoEstimado | 350,00                                               |
      | dataInicio      | 2026-11-16                                           |
      | dataFimPrevista | 2027-02-16                                           |
    Então a resposta tem status 201
```

```
  E o corpo segue o contrato "acao"
  E o campo "categoria" vale "RECICLAGEM"
  E o campo "status" vale "PLANEJADA"
  E o campo "empresaNome" vale o nome da empresa

@positivo
Esquema do Cenário: Atualizar o andamento de uma ação
  Dado que a empresa tem uma ação de "RECICLAGEM" com status "PLANEJADA" e impacto de 350
  Quando eu mudo o status da ação para "<novo status>"
  Então a resposta tem status 200
  E o corpo segue o contrato "acao"
  E o campo "status" vale "<novo status>"

  Exemplos:
    | novo status  |
    | EM_ANDAMENTO |
    | CONCLUIDA    |
    | CANCELADA    |

@positivo
Cenário: Consultar uma ação pelo id
  Dado que a empresa tem uma ação de "EFICIENCIA_ENERGETICA" com status "EM_ANDAMENTO" e impacto de 1200
  Quando eu consulto a ação
  Então a resposta tem status 200
  E o corpo segue o contrato "acao"
  E o campo "titulo" vale "Troca das lâmpadas do galpão por LED"
  E o campo "impactoEstimado" vale 1200

@positivo
Cenário: Listar as ações de uma empresa
  Dado que a empresa tem uma ação de "RECICLAGEM" com status "PLANEJADA" e impacto de 350
  E que a empresa tem uma ação de "REDUCAO_CONSUMO" com status "EM_ANDAMENTO" e impacto de 800
  Quando eu listo as ações da empresa
  Então a resposta tem status 200
  E o corpo segue o contrato "lista-acoes"
  E a lista tem 2 itens

@positivo
Cenário: Filtrar as ações concluídas
  Dado que a empresa tem uma ação de "COMPENSACAO_CARBONO" com status "CONCLUIDA" e impacto de 500
  Quando eu listo as ações com status "CONCLUIDA"
  Então a resposta tem status 200
  E o corpo segue o contrato "lista-acoes"
  E todos os itens têm "status" igual a "CONCLUIDA"
  E a ação cadastrada está na lista

@positivo
Cenário: Ampliar uma ação já cadastrada
  Dado que a empresa tem uma ação de "RECICLAGEM" com status "PLANEJADA" e impacto de 350
  Quando eu altero o título da ação para "Coleta seletiva em todas as unidades" e o impacto para 900
  Então a resposta tem status 200
  E o corpo segue o contrato "acao"
  E o campo "titulo" vale "Coleta seletiva em todas as unidades"
  E o campo "impactoEstimado" vale 900

@positivo
Cenário: Excluir uma ação
  Dado que a empresa tem uma ação de "EDUCACAO_AMBIENTAL" com status "CANCELADA" e impacto de 0
  Quando eu excluo a ação
  Então a resposta tem status 204
  E a ação não é mais encontrada

@negativo
Cenário: Recusar um status que não existe
  Dado que a empresa tem uma ação de "RECICLAGEM" com status "PLANEJADA" e impacto de 350
```

```
    Quando eu mudo o status da ação para "ARQUIVADA"
    Então a resposta tem status 400
    E o corpo segue o contrato "erro"
    E a mensagem de erro é "Corpo da requisição inválido ou mal formatado."

  @negativo @inclusao
  Cenário: Apontar todos os campos obrigatórios que faltam
    Quando eu cadastro uma ação só com a descrição
    Então a resposta tem status 400
    E o corpo segue o contrato "erros-validacao"
    E o campo "titulo" tem o erro "O título é obrigatório"
    E o campo "categoria" tem o erro "A categoria é obrigatória"
    E o campo "status" tem o erro "O status é obrigatório"
    E o campo "dataInicio" tem o erro "A data de início é obrigatória"

  @negativo
  Cenário: Mudar o status de uma ação que não existe
    Quando eu mudo o status da ação de id 999999 para "CONCLUIDA"
    Então a resposta tem status 404
    E o corpo segue o contrato "erro"
    E a mensagem de erro é "Ação ESG não encontrada com ID: 999999"

  @negativo
  Cenário: Consultar uma ação que não existe
    Quando eu consulto "/api/acoes-esg/999999"
    Então a resposta tem status 404
    E o corpo segue o contrato "erro"
    E a mensagem de erro é "Ação ESG não encontrada com ID: 999999"
```

3.4 Relatórios ESG `relatorios.feature`

O contexto monta uma empresa com números conhecidos: dois indicadores e duas ações, com impacto de 300 e 700. Com isso dá para conferir as contas do relatório que vai para a diretoria (1 ação concluída de 2, impacto total de 1000, peso de 30% da ação de eficiência energética) e comparar o resumo geral com as próprias listagens da API.

Cenário	Tipo	O que confere
Resumo de uma empresa	positivo	200, contrato *resumo* e os cinco totais
Peso de uma ação no impacto da empresa	positivo	200, contrato *impacto*, 300 de 1000 = 30%
O resumo geral bate com os registros da API	positivo	totais iguais às listagens de empresas, indicadores e ações
Resumo de uma empresa que não existe	negativo	404
Impacto de uma ação que não existe	negativo	404

`testes/src/test/resources/features/relatorios.feature`

```
# language: pt
@relatorios
Funcionalidade: Relatórios ESG
  Como diretoria da empresa
  Quero ver o resumo dos indicadores e o peso de cada ação no impacto total
  Para prestar contas com números que batem com o que foi lançado
```

```
Contexto:
  Dado que estou autenticado como ADMIN
  E que existe uma empresa cadastrada
  E que a empresa registrou 7800 "kWh" de "ENERGIA"
  E que a empresa registrou 410 "kg" de "RESIDUOS"
  E que a empresa tem uma ação de "EFICIENCIA_ENERGETICA" com status "CONCLUIDA" e impacto de 300
  E que a empresa tem uma ação de "RECICLAGEM" com status "PLANEJADA" e impacto de 700

@positivo
Cenário: Resumo de uma empresa
  Quando eu consulto o resumo da empresa
  Então a resposta tem status 200
  E o corpo segue o contrato "resumo"
  E o resumo mostra:
    | totalEmpresas        | 1    |
    | totalIndicadores     | 2    |
    | totalAcoes           | 2    |
    | acoesConcluidas      | 1    |
    | impactoTotalEstimado | 1000 |

@positivo
Cenário: Peso de uma ação no impacto da empresa
  Quando eu consulto o impacto da ação de "EFICIENCIA_ENERGETICA"
  Então a resposta tem status 200
  E o corpo segue o contrato "impacto"
  E o campo "impactoEstimado" vale 300
  E o campo "impactoTotalEmpresa" vale 1000
  E o campo "percentualSobreTotal" vale 30

@positivo @governanca
Cenário: O resumo geral bate com os registros da API
  Quando eu consulto o resumo geral
  Então a resposta tem status 200
  E o corpo segue o contrato "resumo"
  E os totais do resumo geral batem com as listagens de empresas, indicadores e ações

@negativo
Cenário: Resumo de uma empresa que não existe
  Quando eu consulto o resumo da empresa de id 999999
  Então a resposta tem status 404
  E o corpo segue o contrato "erro"
  E a mensagem de erro é "Empresa não encontrada com ID: 999999"

@negativo
Cenário: Impacto de uma ação que não existe
  Quando eu consulto o impacto da ação de id 999999
  Então a resposta tem status 404
  E o corpo segue o contrato "erro"
  E a mensagem de erro é "Ação ESG não encontrada com ID: 999999"
```

3.5 Acesso por perfil `acesso.feature`

A leitura dos dados ESG é pública, mas só o perfil ADMIN altera registros. As consultas abertas cadastram antes uma empresa, um indicador e uma ação, para o contrato da lista validar itens de verdade, e precisam responder sem login e em menos de 2 segundos. Qualquer escrita sem login, com senha errada ou com o perfil USER é recusada.

Cenário	Tipo	O que confere
Consultar dados públicos sem login (4 exemplos)	positivo	200, o contrato de cada rota (três listas e o resumo) e tempo de resposta abaixo de 2000 ms

Cenário	Tipo	O que confere
Perfil USER consulta os relatórios	positivo	200
Cadastro sem login	negativo	401 com o cabeçalho WWW-Authenticate: Basic
Cadastro com senha errada	negativo	401
Perfil USER não altera dados (4 exemplos)	negativo	403 em POST, PUT, PATCH e DELETE, contrato *acesso-negado*

testes/src/test/resources/features/acesso.feature

```
# language: pt
@acesso @governanca
Funcionalidade: Acesso por perfil
  Como responsável pela governança dos dados ESG
  Quero que só o perfil ADMIN altere registros
  Para que os números publicados não sejam mudados por quem não tem permissão

  @positivo @eficiencia
  Esquema do Cenário: Consultar dados públicos sem login
    Dado que existe uma empresa cadastrada
    E que a empresa registrou 100 "m3" de "AGUA"
    E que a empresa tem uma ação de "RECICLAGEM" com status "PLANEJADA" e impacto de 350
    E que não estou autenticado
    Quando eu consulto "<endereço>"
    Então a resposta tem status 200
    E o corpo segue o contrato "<contrato>"
    E a resposta chega em menos de 2000 ms

    Exemplos:
      | endereço               | contrato          |
      | /api/empresas          | lista-empresas    |
      | /api/indicadores       | lista-indicadores |
      | /api/acoes-esg         | lista-acoes       |
      | /api/relatorios/resumo | resumo            |

  @positivo
  Cenário: Perfil USER consulta os relatórios
    Dado que estou autenticado como USER
    Quando eu consulto "/api/relatorios/resumo"
    Então a resposta tem status 200
    E o corpo segue o contrato "resumo"

  @negativo
  Cenário: Cadastro sem login
    Dado que não estou autenticado
    Quando eu tento cadastrar uma empresa
    Então a resposta tem status 401
    E a resposta pede autenticação Basic

  @negativo
  Cenário: Cadastro com senha errada
    Dado que estou autenticado como ADMIN com a senha "senha-errada"
    Quando eu tento cadastrar uma empresa
    Então a resposta tem status 401
    E a resposta pede autenticação Basic

  @negativo
  Esquema do Cenário: Perfil USER não altera dados
    Dado que estou autenticado como USER
```

```
    Quando eu envio <método> para "<endereço>"
    Então a resposta tem status 403
    E o corpo segue o contrato "acesso-negado"

    Exemplos:
      | método | endereço                |
      | POST   | /api/empresas           |
      | PUT    | /api/indicadores/1      |
      | PATCH  | /api/acoes-esg/1/status |
      | DELETE | /api/empresas/1         |
```

3.6 Saúde e versão da API `monitoramento.feature`

O Docker, o pipeline e a equipe de operação usam health e info para saber se a API está no ar e o que está rodando. O contrato do info exige a versão começando por X.Y.Z e o commit como hash de 7 a 40 caracteres, ou local quando a imagem foi gerada na máquina.

Cenário	Tipo	O que confere
API e banco no ar	positivo	200, status UP da API e do banco
Versão e commit da imagem publicados	positivo	200 e contrato *info*

testes/src/test/resources/features/monitoramento.feature

```
# language: pt
@monitoramento @governanca
Funcionalidade: Saúde e versão da API
  Como equipe de operação
  Quero saber se a API e o banco estão no ar e qual versão está rodando
  Para rastrear o que foi publicado em cada ambiente

  Contexto:
    Dado que não estou autenticado

  @positivo
  Cenário: API e banco no ar
    Quando eu consulto "/actuator/health"
    Então a resposta tem status 200
    E o corpo segue o contrato "health"
    E o campo "status" vale "UP"
    E o campo "components.db.status" vale "UP"

  @positivo
  Cenário: Versão e commit da imagem publicados
    Quando eu consulto "/actuator/info"
    Então a resposta tem status 200
    E o corpo segue o contrato "info"
    E o campo "app.nome" vale "EcoTrack API"
```

3.7 Dos passos ao código

Cada frase do Gherkin corresponde a um método anotado. Os passos "Então" comuns a todas as rotas (status, contrato, campo, mensagem de erro) ficam em `RespostaSteps`. Abaixo, o passo que valida o contrato, a classe que faz a validação com o JSON Schema e o método que cria os dados de apoio e marca o registro para ser apagado no fim do cenário.

RespostaSteps.java

```
    @Então("o corpo segue o contrato {string}")
    public void oCorpoSegueOContrato(String contrato) {
        assertThat(Contrato.violacoes(contrato, cenario.resposta().asString()))
                .as("violações do contrato %s.json", contrato)
                .isEmpty();
    }
```

Contrato.java

```
package br.com.fiap.ecotrack.bdd;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.networknt.schema.JsonSchema;
import com.networknt.schema.JsonSchemaFactory;
import com.networknt.schema.PathType;
import com.networknt.schema.SchemaLocation;
import com.networknt.schema.SchemaValidatorsConfig;
import com.networknt.schema.SpecVersion;
import com.networknt.schema.ValidationMessage;

import java.util.List;

// Contratos das respostas em src/test/resources/schemas (JSON Schema 2020-12).
// O "$ref" entre arquivos é resolvido pelo classpath.
final class Contrato {

    private static final JsonSchemaFactory FABRICA =
JsonSchemaFactory.getInstance(SpecVersion.VersionFlag.V202012);
    // no draft 2020-12 o "format" é só anotação; ligado, "2026-13-45" deixa de passar como data.
    // JSON_PATH deixa o campo da violação no formato "$.dataCadastro".
    private static final SchemaValidatorsConfig CONFIG = SchemaValidatorsConfig.builder()
            .formatAssertionsEnabled(true)
            .pathType(PathType.JSON_PATH)
            .build();
    private static final ObjectMapper JSON = new ObjectMapper();

    private Contrato() {
    }

    static List<String> violacoes(String contrato, String corpo) {
        JsonSchema schema = FABRICA.getSchema(SchemaLocation.of("classpath:schemas/" + contrato +
".json"), CONFIG);
        try {
            return schema.validate(JSON.readTree(corpo)).stream()
                    .map(ValidationMessage::getMessage)
                    .sorted()
                    .toList();
        } catch (JsonProcessingException e) {
            return List.of("a resposta não é JSON: " + corpo);
        }
    }
}
```

Cenario.java

```
    // cadastro de apoio para o "Dado", sempre com o ADMIN
    Registro criar(String caminho, Map<String, Object> dados) {
        Response criado = requisicao(Api.ADMIN).body(dados).post(caminho);
        assertThat(criado.statusCode())
                .as("cadastro de apoio em %s respondeu %s", caminho, criado.asString())
                .isEqualTo(201);
```

```
    long id = criado.jsonPath().getLong("id");
    criados.push(caminho + "/" + id);
    return new Registro(id, dados);
}

void apagarCriados() {
    while (!criados.isEmpty()) {
        RestAssured.given()
                .baseUri(Api.URL)
                .auth().preemptive().basic(Api.ADMIN.usuario(), Api.ADMIN.senha())
                .delete(criados.pop());
    }
}
```

4. Testes de API e contratos

Todas as rotas passam por pelo menos um cenário que confere o status. Quando a resposta tem corpo, o cenário também confere os campos e o contrato. A tabela mostra os status testados em cada rota e os contratos usados.

Rota	Status testados	Contratos
`GET /api/empresas`	200	lista-empresas
`GET /api/empresas/{id}`	200, 400, 404	empresa, erro
`POST /api/empresas`	201, 400, 401, 403	empresa, erro, erros-validacao, acesso-negado
`PUT /api/empresas/{id}`	200	empresa
`DELETE /api/empresas/{id}`	204, 403, 409	erro, acesso-negado
`GET /api/indicadores`	200, 400	lista-indicadores, erro
`GET /api/indicadores/{id}`	200, 404	indicador, erro
`GET /api/indicadores/empresa/{id}`	200	lista-indicadores
`POST /api/indicadores`	201, 400, 404	indicador, erros-validacao, erro
`PUT /api/indicadores/{id}`	200, 403	indicador, acesso-negado
`DELETE /api/indicadores/{id}`	204	sem corpo
`GET /api/acoes-esg`	200	lista-acoes
`GET /api/acoes-esg/{id}`	200, 404	acao, erro
`GET /api/acoes-esg/empresa/{id}`	200	lista-acoes
`POST /api/acoes-esg`	201, 400	acao, erros-validacao
`PUT /api/acoes-esg/{id}`	200	acao
`PATCH /api/acoes-esg/{id}/status`	200, 400, 403, 404	acao, erro, acesso-negado
`DELETE /api/acoes-esg/{id}`	204	sem corpo
`GET /api/relatorios/resumo`	200	resumo
`GET /api/relatorios/empresa/{id}`	200, 404	resumo, erro
`GET /api/relatorios/impacto/{id}`	200, 404	impacto, erro
`GET /actuator/health`	200	health
`GET /actuator/info`	200	info

Como os contratos foram escritos

- JSON Schema draft 2020-12, validado com a biblioteca networknt json-schema-validator. Os arquivos ficam em `src/test/resources/schemas`.
- Nos contratos dos DTOs da API (empresa, indicador, acao, impacto, resumo, erro e erros-validacao) todos os campos estão em `required` e os objetos têm `additionalProperties: false`: se um campo for removido, renomeado ou aparecer um campo novo, o teste falha. Os que podem ser nulos (`observacao`, `impactoEstimado` e `dataFimPrevista`) têm `null` entre os tipos aceitos.
- Os contratos do Actuator (health e info) e do 403 (acesso-negado) exigem só os campos que os testes usam.
- Enums com os valores de `TipoIndicador`, `CategoriaAcao` e `StatusAcao`; datas no formato AAAA-MM-DD; CNPJ com máscara; UF com duas letras maiúsculas; percentual entre 0 e 100.
- Tipos repetidos ficam em `comum.json`, e as listas apontam para o contrato do item com `$ref`.
- A API tem três formatos de erro, cada um com o seu contrato: o objeto do `GlobalExceptionHandler` (status, mensagem, dataHora), a lista de erros de validação (`[{campo, mensagem}]`) e o 403 com o corpo padrão do Spring Boot (timestamp, status, error, path). O 401 vem sem corpo.
- O validador roda com a checagem de `format` ligada, então uma data como 2026-13-45 não passa.

Para conferir que o contrato pega uma mudança de verdade, tirei o campo `dataCadastro` de uma cópia do `empresa.json` e rodei os 5 cenários positivos de empresa (`@empresas and @positivo`): os 4 que validam esse contrato falharam e mostraram a violação (print 05, na seção 7.4).

schemas/empresa.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Empresa",
  "type": "object",
  "required": ["id", "nome", "cnpj", "setor", "cidade", "estado", "dataCadastro"],
  "additionalProperties": false,
  "properties": {
    "id": { "$ref": "comum.json#/$defs/id" },
    "nome": { "$ref": "comum.json#/$defs/texto", "maxLength": 150 },
    "cnpj": { "type": "string", "pattern": "^\\d{2}\\.\\d{3}\\.\\d{3}/\\d{4}-\\d{2}$" },
    "setor": { "$ref": "comum.json#/$defs/texto", "maxLength": 100 },
    "cidade": { "$ref": "comum.json#/$defs/texto", "maxLength": 100 },
    "estado": { "type": "string", "pattern": "^[A-Z]{2}$" },
    "dataCadastro": { "$ref": "comum.json#/$defs/data" }
  }
}
```

schemas/lista-empresas.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Lista de empresas",
  "type": "array",
  "items": { "$ref": "empresa.json" }
}
```

schemas/acao.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Ação ESG",
```

```
  "type": "object",
  "required": ["id", "empresaId", "empresaNome", "titulo", "descricao", "categoria", "status",
"impactoEstimado", "dataInicio", "dataFimPrevista"],
  "additionalProperties": false,
  "properties": {
    "id": { "$ref": "comum.json#/$defs/id" },
    "empresaId": { "$ref": "comum.json#/$defs/id" },
    "empresaNome": { "$ref": "comum.json#/$defs/texto" },
    "titulo": { "$ref": "comum.json#/$defs/texto", "maxLength": 150 },
    "descricao": { "$ref": "comum.json#/$defs/texto", "maxLength": 500 },
    "categoria": { "$ref": "comum.json#/$defs/categoriaAcao" },
    "status": { "$ref": "comum.json#/$defs/statusAcao" },
    "impactoEstimado": { "type": ["number", "null"], "minimum": 0 },
    "dataInicio": { "$ref": "comum.json#/$defs/data" },
    "dataFimPrevista": {
      "anyOf": [{ "$ref": "comum.json#/$defs/data" }, { "type": "null" }]
    }
  }
}
```

schemas/comum.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Tipos usados por mais de um contrato",
  "$defs": {
    "id": { "type": "integer", "minimum": 1 },
    "data": { "type": "string", "format": "date", "pattern": "^\\d{4}-\\d{2}-\\d{2}$" },
    "quantia": { "type": "number", "minimum": 0 },
    "texto": { "type": "string", "minLength": 1 },
    "tipoIndicador": { "enum": ["AGUA", "ENERGIA", "RESIDUOS", "EMISSAO_CO2"] },
    "categoriaAcao": { "enum": ["REDUCAO_CONSUMO", "RECICLAGEM", "COMPENSACAO_CARBONO",
"EFICIENCIA_ENERGETICA", "EDUCACAO_AMBIENTAL"] },
    "statusAcao": { "enum": ["PLANEJADA", "EM_ANDAMENTO", "CONCLUIDA", "CANCELADA"] }
  }
}
```

5. ESG nos testes

Os testes cobrem as regras que fazem o EcoTrack servir para prestação de contas ESG. A tabela liga cada pilar aos cenários.

Pilar	Onde aparece nos testes
Ambiental	Registro e correção de indicadores de energia, água, resíduos e CO2. Ações de reciclagem (coleta seletiva), eficiência energética (troca por LED) e compensação de carbono. Tempo de resposta das consultas públicas abaixo de 2 s (`@eficiencia`), ligado ao exemplo de eficiência e uso de recursos computacionais do enunciado.
Social	Mensagens de erro em português que apontam o campo e o motivo, para quem usa a API corrigir o envio sem precisar ler o código (`@inclusao`). Leitura pública dos dados e dois perfis de uso, ADMIN e USER.
Governança	Só o ADMIN altera dados (401 e 403). CNPJ único. Empresa com indicadores não pode ser apagada (409). O resumo geral bate com as listagens. Health e info mostram a versão e o commit em execução. Os contratos travam mudanças na API que não foram combinadas com quem consome os dados.

6. Como executar

Só com Docker

Precisa apenas do Docker com o Compose v2. O script sobe o Oracle e a API, espera os dois ficarem healthy, roda os cenários num container com Maven e JDK 21 e derruba o ambiente. O relatório fica em `testes/target/cucumber/relatorio.html`.

terminal

```
scripts/executar-testes.sh

# só uma parte dos cenários, pelo filtro de tags do Cucumber
scripts/executar-testes.sh -Dcucumber.filter.tags=@negativo
scripts/executar-testes.sh -Dcucumber.filter.tags="@empresas and not @negativo"

# os mesmos passos, separados
docker compose up -d --wait oracle api   # API em http://localhost:8080
docker compose run --rm --build testes   # roda os cenários (o --build pega mudanças nos testes)
docker compose down -v                   # derruba e apaga o banco
```

Com Maven na máquina

terminal

```
docker compose up -d --wait oracle api
cd testes
mvn test

# contra outro ambiente, por exemplo o staging do projeto de DevOps
mvn test -DECOTRACK_BASE_URL=http://localhost:8081 -DECOTRACK_ADMIN_PASSWORD=<senha>
```

Variável ou -D	Padrão
`ECOTRACK_BASE_URL`	`http://localhost:8080`
`ECOTRACK_ADMIN_USER` / `ECOTRACK_ADMIN_PASSWORD`	`admin@ecotrack.com` / `admin123`
`ECOTRACK_USER` / `ECOTRACK_USER_PASSWORD`	`user@ecotrack.com` / `user123`
`ECOTRACK_IMAGE` (docker compose)	`ghcr.io/psouzah12/ecotrack-api:sha-92e76ea`

No GitHub Actions

O repositório é github.com/psouzah12/ecotrack-testes. O workflow `.github/workflows/testes-bdd.yml` roda em push para `main` (menos quando o push só muda arquivos .md, `docs/` ou `evidencias/`), em pull request para `main` e pelo botão "Run workflow", que tem um campo para filtrar por tag. Ele usa o mesmo `docker-compose.yml` para subir o Oracle e a API, roda o `mvn test` com o JDK 21 do runner, escreve na aba Summary uma tabela de cenários por funcionalidade (gerada do `cucumber.json` pelo `scripts/ci/resumo-cucumber.sh`) e publica o relatório do Cucumber como artefato. Se algo falha, mostra os logs da API. O ambiente é derrubado sempre, no último passo.

7. Evidências

Execução feita em 06/11/2026, às 22h00, num Mac com Docker 29 (arm64), a partir do ambiente zerado (`docker compose down -v` antes). Rodei o script com `MANTER_AMBIENTE=1` para o Oracle e a API continuarem no ar nas evidências 7.3 a 7.5 e derrubei o ambiente no fim. Os prints de terminal foram montados a partir dos logs completos, que estão em `evidencias/logs`; na saída do Cucumber tirei só a coluna com o nome do método Java de cada passo, para caber na largura. O relatório HTML completo está em `evidencias/relatorio-cucumber/relatorio.html`.

7.1 Execução completa com Docker

01 - scripts/executar-testes.sh: Oracle e API sobem e o container de testes compila

```
$ MANTER_AMBIENTE=1 scripts/executar-testes.sh
==> Subindo Oracle e API (na primeira vez leva uns 2 minutos)
 Network ecotrack-testes_default Creating
 Network ecotrack-testes_default Created
 Container ecotrack-testes-oracle-1 Creating
 Container ecotrack-testes-oracle-1 Created
 Container ecotrack-testes-api-1 Creating
 Container ecotrack-testes-api-1 Created
 Container ecotrack-testes-oracle-1 Starting
 Container ecotrack-testes-oracle-1 Started
 Container ecotrack-testes-oracle-1 Waiting
 Container ecotrack-testes-oracle-1 Healthy
 Container ecotrack-testes-api-1 Starting
 Container ecotrack-testes-api-1 Started
 Container ecotrack-testes-oracle-1 Waiting
 Container ecotrack-testes-api-1 Waiting
 Container ecotrack-testes-oracle-1 Healthy
 Container ecotrack-testes-api-1 Healthy
SERVICE   IMAGE                                      STATUS
api       ghcr.io/psouzah12/ecotrack-api:sha-92e76ea   Up 10 seconds (healthy)
oracle    gvenzl/oracle-free:23-slim                   Up 21 seconds (healthy)
==> API em http://localhost:8080
... build da imagem dos testes ...
[INFO] Compiling 12 source files with javac [debug release 21] to target/test-classes
[INFO]  T E S T S
[INFO] Running br.com.fiap.ecotrack.bdd.ExecutarCenariosTest
```

O script sobe o Oracle e a API, espera os dois ficarem healthy e compila os testes no container.

02 - Um cenário no console: passos do Gherkin e as requisições feitas à API

```
@empresas @negativo @governanca
Cenário: Recusar um CNPJ que já está cadastrado
  ✔ Dado que estou autenticado como ADMIN
  ✔ Dado que existe uma empresa cadastrada
  ✔ Quando eu cadastro outra empresa com o mesmo CNPJ
  ✔ Então a resposta tem status 400
  ✔ E o corpo segue o contrato "erro"
  ✔ E a mensagem de erro é "Já existe uma empresa cadastrada com este CNPJ."
        POST http://api:8080/api/empresas
        {"nome":"Verde Vale Reciclagem","cnpj":"98.274.558/5063-98","setor":"Reciclagem","cidade":"Santos","estado":"S
P"}
        => HTTP/1.1 201  (79 ms)
        {
            "id": 26,
            "nome": "Verde Vale Reciclagem",
            "cnpj": "98.274.558/5063-98",
            "setor": "Reciclagem",
            "cidade": "Santos",
            "estado": "SP",
            "dataCadastro": "2026-11-06"
        }
        POST http://api:8080/api/empresas
        {"nome":"Outra Empresa LTDA","cnpj":"98.274.558/5063-98","setor":"Reciclagem","cidade":"Santos","estado":"SP"}
        => HTTP/1.1 400  (77 ms)
        {
            "status": 400,
            "mensagem": "Já existe uma empresa cadastrada com este CNPJ.",
            "dataHora": "2026-11-06T21:59:58.655959836"
```

Um cenário no console. Abaixo dos passos ficam as requisições que ele fez: o cadastro de apoio (201) e o cadastro com o CNPJ repetido (400).

03 - Fim da execução completa: 56 cenários e 436 passos

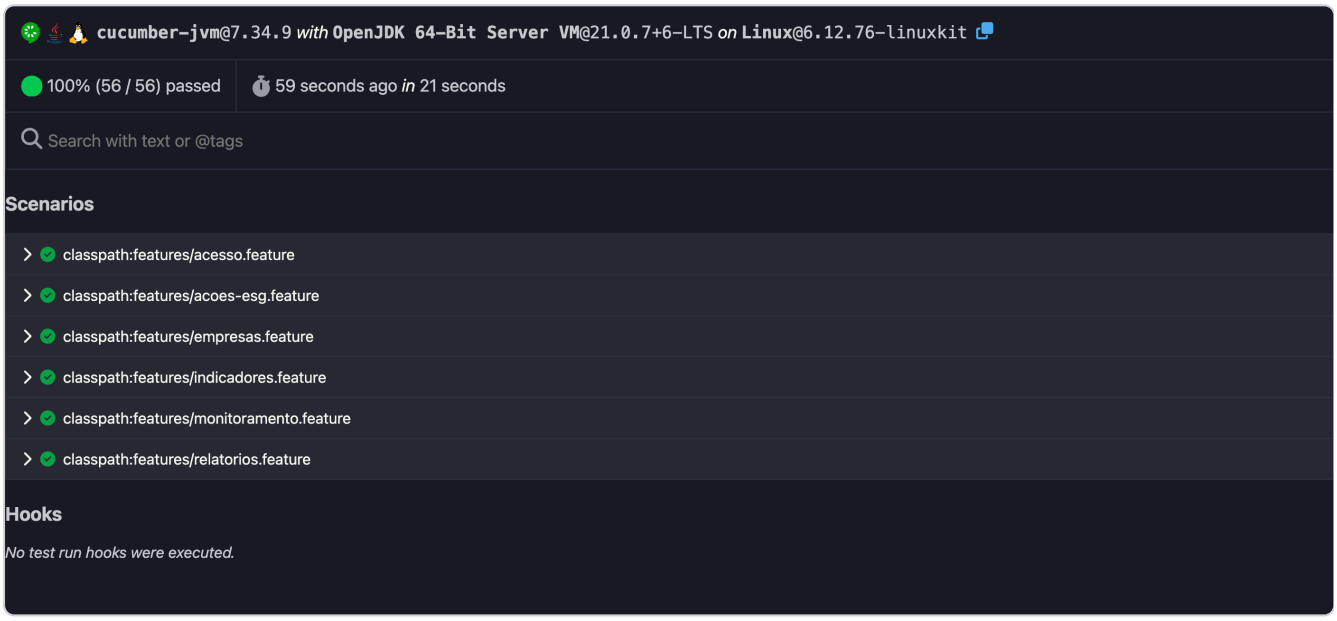
```
... 56 cenários rodados (log completo em evidencias/logs/01-execucao-docker.log) ...
56 scenarios (56 passed)
436 steps (436 passed)
0m 21.121s
[INFO] Tests run: 56, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 21.38 s -- in br.com.fiap.ecotrack.bdd.Executa
rCenariosTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 56, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  23.129 s
[INFO] Finished at: 2026-11-07T01:00:09Z
[INFO] ------------------------------------------------------------------------
 Container ecotrack-testes-testes-run-5bb6ec118107 Creating
 Container ecotrack-testes-testes-run-5bb6ec118107 Created
==> Relatório em testes/target/cucumber/relatorio.html
exit=0
```

Fim da execução: 56 cenários e 436 passos, todos passando.

Funcionalidade	Arquivo	Cenários	Passaram	Falharam
Acesso por perfil	`acesso.feature`	11	11	0
Ações ESG	`acoes-esg.feature`	13	13	0
Cadastro de empresas	`empresas.feature`	12	12	0

Funcionalidade	Arquivo	Cenários	Passaram	Falharam
Indicadores ambientais	`indicadores.feature`	13	13	0
Saúde e versão da API	`monitoramento.feature`	2	2	0
Relatórios ESG	`relatorios.feature`	5	5	0

7.2 Relatório HTML do Cucumber



Visão geral do relatório: 100% (56 de 56) e um item por funcionalidade.



Funcionalidade de relatórios aberta, com a tabela do resumo esperado e os campos do impacto.



Anexo de um cenário com as requisições e respostas reais da API.

7.3 Filtro por tag

04 - Só os cenários @negativo, filtrando por tag

```
$ docker compose run --rm --build testes -Dcucumber.filter.tags="@negativo"
...
23 scenarios (23 passed)
157 steps (157 passed)
0m 6.534s
[WARNING] Tests run: 56, Failures: 0, Errors: 0, Skipped: 33, Time elapsed: 6.744 s -- in br.com.fiap.ecotrack.bdd.Exe
cutarCenariosTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 56, Failures: 0, Errors: 0, Skipped: 33
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  7.523 s
```

Só os 23 cenários @negativo, pelo docker compose. O Surefire conta os 33 de fora do filtro como skipped.

7.4 O contrato pega mudança na API

05 - Contrato quebrado de propósito: o teste falha e mostra a violação

```
$ # cópia do empresa.json sem o campo dataCadastro (alterada só em target/test-classes)
$ mvn -B surefire:test -Dcucumber.filter.tags="@empresas and @positivo"
...
@empresas @positivo
Cenário: Cadastrar uma empresa
  ✔ Dado que estou autenticado como ADMIN
  ✔ Quando eu cadastro a empresa:
      | nome   | Reciclar Embalagens LTDA |
      | setor  | Reciclagem               |
      | cidade | Santos                   |
      | estado | sp                       |
  ✔ Então a resposta tem status 201
  ✘ E o corpo segue o contrato "empresa"
        java.lang.AssertionError: [violações do contrato empresa.json]
        Expecting empty but was: ["$: a propriedade 'dataCadastro' não está definida no esquema e o esquema não permit
e propriedades adicionais"]
                at br.com.fiap.ecotrack.bdd.RespostaSteps.oCorpoSegueOContrato(RespostaSteps.java:32)
                at ✱.o corpo segue o contrato "empresa"(classpath:features/empresas.feature:19)
  ↷ E o campo "nome" vale "Reciclar Embalagens LTDA"
  ↷ E o campo "estado" vale "SP"
  ↷ E a empresa aparece na listagem de empresas
...
5 scenarios (1 passed, 4 failed)
36 steps (25 passed, 7 skipped, 4 failed)
[ERROR] Tests run: 56, Failures: 4, Errors: 0, Skipped: 51
[INFO] BUILD FAILURE
```

Com o dataCadastro tirado de uma cópia do empresa.json, 4 dos 5 cenários positivos de empresa falham e mostram a violação. O de exclusão passa, porque só valida o 404 com o contrato erro.

7.5 Ambiente depois dos testes

06 - Ambiente depois dos testes: containers healthy, versão no ar e banco limpo

```
$ docker compose ps --format 'table {{.Service}}\t{{.Image}}\t{{.Status}}'
SERVICE   IMAGE                                    STATUS
api       ghcr.io/psouzah12/ecotrack-api:sha-92e76ea   Up 47 seconds (healthy)
oracle    gvenzl/oracle-free:23-slim               Up 58 seconds (healthy)

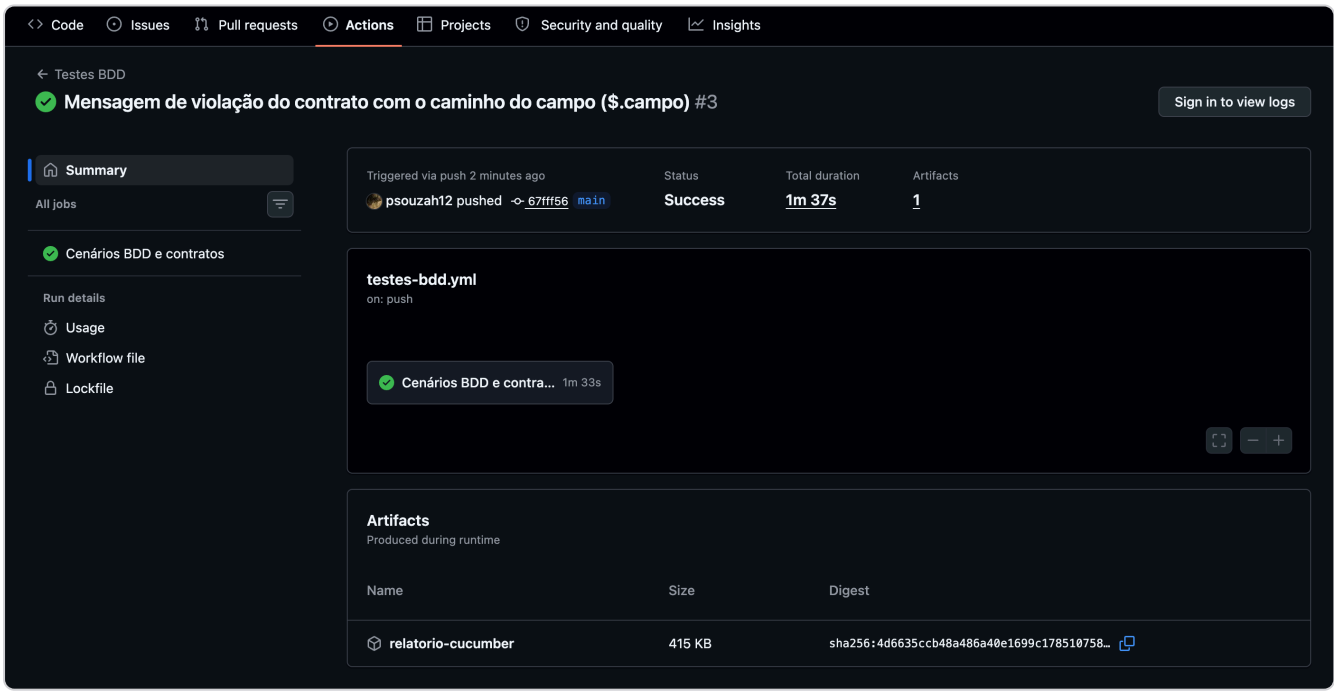
$ curl -s http://localhost:8080/actuator/info
{"app":{"nome":"EcoTrack API","descricao":"Gestao de indicadores e acoes ESG","versao":"1.1.0","ambiente":"testes","co
mmit":"92e76ea5ffda0f5a00542f2721d994720389ec69"},"build":{"artifact":"ecotrack-api","name":"EcoTrack API","time":"202
6-10-14T01:02:48.994Z","version":"1.1.0","group":"br.com.fiap"},"java":{"version":"21.0.12.1","vendor":{"name":"Eclips
e Adoptium","version":"Temurin-21.0.12.1+1"},"runtime":{"name":"OpenJDK Runtime Environment","version":"21.0.12.1+1-LT
S"},"jvm":{"name":"OpenJDK 64-Bit Server VM","vendor":"Eclipse Adoptium","version":"21.0.12.1+1-LTS"}}}

$ curl -s http://localhost:8080/api/relatorios/resumo   # depois dos testes sobram só os dados de exemplo
{"totalEmpresas":3,"totalIndicadores":4,"totalAcoes":3,"acoesConcluidas":0,"impactoTotalEstimado":2550}
```

Containers healthy, versão 1.1.0 e commit 92e76ea no /actuator/info. O resumo voltou aos números dos dados de exemplo (3 empresas, 4 indicadores, 3 ações): os testes apagaram tudo o que criaram.

7.6 Execução no GitHub Actions

O workflow *Testes BDD* roda a cada push em `main`. Todos os runs passaram: o #1 com a primeira versão (53 cenários) e os seguintes com os ajustes feitos depois de uma revisão. O run #3 é o da versão entregue: github.com/psouzah12/ecotrack-testes/actions/runs/42321464083. O job subiu o Oracle e a API no runner `ubuntu-24.04`, rodou os 56 cenários e publicou o relatório como artefato. A página do run é pública; os logs e a aba Summary pedem login no GitHub, então os prints 07 e 08 são trechos do log do job baixado pela API do GitHub (`evidencias/logs/github`), e a tabela do resumo foi montada pelo mesmo `resumo-cucumber.sh` a partir do `cucumber.json` do artefato.



Run #3 no GitHub Actions: disparado pelo push em main, status Success, 1m37s no total e o artefato relatorio-cucumber.

Cenários BDD e contratos summary

Cenários BDD

Funcionalidade	Cenários	Passaram	Falharam
Acesso por perfil	11	11	0
Ações ESG	13	13	0
Cadastro de empresas	12	12	0
Indicadores ambientais	13	13	0
Saúde e versão da API	2	2	0
Relatórios ESG	5	5	0

Todos os cenários passaram.

Tabela de cenários por funcionalidade que o workflow escreve na aba Summary.

07 - GitHub Actions, passo "Subir Oracle e API" (run #3)

```
$ docker compose up -d --wait oracle api
 Network ecotrack-testes_default  Creating
 Network ecotrack-testes_default  Created
 Container ecotrack-testes-oracle-1  Creating
 Container ecotrack-testes-oracle-1  Created
 Container ecotrack-testes-api-1  Creating
 Container ecotrack-testes-api-1  Created
 Container ecotrack-testes-oracle-1  Starting
 Container ecotrack-testes-oracle-1  Started
 Container ecotrack-testes-oracle-1  Waiting
 Container ecotrack-testes-oracle-1  Healthy
 Container ecotrack-testes-api-1  Starting
 Container ecotrack-testes-api-1  Started
 Container ecotrack-testes-oracle-1  Waiting
 Container ecotrack-testes-api-1  Waiting
 Container ecotrack-testes-oracle-1  Healthy
 Container ecotrack-testes-api-1  Healthy
$ docker compose ps
NAME                      IMAGE                                        COMMAND                  SERVICE   CREATED
STATUS                    PORTS
ecotrack-testes-api-1     ghcr.io/psouzah12/ecotrack-api:sha-92e76ea   "java org.springfram…"   api       32 seconds
ago   Up 10 seconds (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
ecotrack-testes-oracle-1   gvenzl/oracle-free:23-slim                  "container-entrypoin…"   oracle    32 seconds
ago   Up 31 seconds (healthy)
$ curl -s http://localhost:8080/actuator/info
{"app":{"nome":"EcoTrack API","descricao":"Gestao de indicadores e acoes ESG","versao":"1.1.0","ambiente":"testes","co
mmit":"92e76ea5ffda0f5a00542f2721d994720389ec69"},"build":{"artifact":"ecotrack-api","name":"EcoTrack API","time":"202
6-10-14T01:02:48.994Z","version":"1.1.0","group":"br.com.fiap"},"java":{"version":"21.0.12.1","vendor":{"name":"Eclips
e Adoptium","version":"Temurin-21.0.12.1+1"},"runtime":{"name":"OpenJDK Runtime Environment","version":"21.0.12.1+1-LT
S"},"jvm":{"name":"OpenJDK 64-Bit Server VM","vendor":"Eclipse Adoptium","version":"21.0.12.1+1-LTS"}}}
```

Passo que sobe o Oracle e a API no runner com o mesmo docker-compose.yml do projeto.

08 - GitHub Actions, passo "Rodar os cenários" (run #3)

```
$ cd testes && mvn -B -ntp test
... cenários contra http://localhost:8080 (log completo em evidencias/logs/github) ...
56 scenarios (56 passed)
436 steps (436 passed)
0m 20.260s
[INFO] Tests run: 56, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 20.59 s -- in br.com.fiap.ecotrack.bdd.Executa
rCenariosTest
[INFO] Results:
[INFO] Tests run: 56, Failures: 0, Errors: 0, Skipped: 0
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  25.300 s
[INFO] Finished at: 2026-11-07T01:00:30Z
```

Passo que roda os cenários: 56 de 56 e 436 passos.

8. O que os testes mostraram

Escrevendo os cenários, encontrei três comportamentos da API que não transformei em teste com falha, porque a API nunca prometeu essas regras. Acho que elas devem entrar na próxima versão. Conferi cada um com chamadas diretas, que estão em `evidencias/logs/05-comportamentos-api.log`.

Comportamento	O que fazer
A API aceita uma ação com `dataFimPrevista` anterior à `dataInicio` (testei com início em 01/12/2026 e fim em 01/01/2026: 201).	Validar a ordem das datas no `AcaoRequest` e devolver 400.
O status de uma ação volta de CONCLUIDA para PLANEJADA sem nenhum registro da mudança.	Definir as transições permitidas e guardar o histórico de status, que é o tipo de rastro que uma auditoria ESG pede.
O 403 e o 401 não seguem o formato de erro da API: o 403 vem com o corpo padrão do Spring Boot (timestamp, status, error, path) e o 401 vem sem corpo.	Configurar no `SecurityConfig` um `accessDeniedHandler` e um `authenticationEntryPoint` que escrevam o mesmo `ErroResponse`. Hoje os testes precisam de um contrato só para o 403 por causa disso.